



Overview

Students learn to play sound effects and music in Phaser, and to control them. The Phaser ideas are **the sound manager** (`this.sound`), the difference between **fire-and-forget** `play()` and a kept sound object from `add()` , **sound configs** (volume, rate, detune, pan, loop, delay), a sound's **states and events**, **cross-fading with tweens**, and the browser's **autoplay rule**. The one big idea - the one that causes the bugs - is that **the sound manager belongs to the game, not to a scene**: sounds outlive the scene that started them, and so do listeners on the sound manager. It connects directly to Chapter 2 (scenes are reused) and Chapter 4 (clean up listeners at `SHUTDOWN`).

The chapter is fun to teach - students like making noise - but it is also the first chapter where the results cannot be seen, only heard. Headphones for everyone make the lab much calmer, and are needed to hear panning at all.

Prerequisites

- Chapter 2: scenes, `this.scene.start` , `init()` and the reused-scene trap, the registry, object literals, spread and union types
- Chapter 3: loading assets in `preload()`
- Chapter 4: scene events, especially `SHUTDOWN` , and removing listeners on emitters that outlive a scene
- Tweens are used lightly (fading a volume); Chapter 7 covers them properly. One sentence of explanation is enough here

Learning outcomes

Students can:

1. load sounds and play them with `this.sound.play(key, config)` , choosing volume, rate, detune, pan, loop and delay

2. choose between `this.sound.play()` and `this.sound.add()` , and destroy sounds they `add()`
3. control a playing sound - pause, resume, stop, seek, change its settings - and react to its `"complete"` event
4. explain why music carries on across scenes, and use `this.sound.get(key)` to avoid starting a second copy
5. fade and cross-fade music with tweens, and explain why the tweens must belong to the scene that is starting
6. mute or change the volume of the whole game, and explain the browser's autoplay rule and `this.sound.locked`

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Play <code>ch05_sound_board</code> on the projector (speakers on). Press a pad ten times: "what is wrong with that?" (it sounds like a machine). Then press R and do it again
0:10 - 0:30	Slides 3-7: loading, <code>play</code> with a config, rate and detune, the seconds trap, random pitch and pan. Students try every key on the sound board
0:30 - 0:45	Slides 8-10: <code>add</code> versus <code>play</code> , <code>"complete"</code> and <code>destroy</code> , the three states. Walk through <code>SoundPad.play()</code> and <code>playOrPauseMusic()</code> . Challenge 1
0:45 - 1:00	Live demo of the duplicate music bug (below). Slides 11-12: the game-wide sound manager, <code>this.sound.get</code>
1:00 - 1:15	Slides 13-15: <code>Music.play</code> , cross-fades, whose tweens, mute. Students play <code>ch05_music_across_scenes</code> and watch the monitor line while moving between scenes
1:15 - 1:25	Slides 16-18: autoplay, <code>locked</code> , the title scene - and the listener bug it had
1:25 - 2:00	Challenges 2-6

Key points to stress

- `this.sound` **is the same object in every scene**. Draw the game-wide diagram on the board. Sounds keep playing when a scene shuts down; that is a feature for music and a bug for everything else
- `play()` **for effects**, `add()` **for anything you need to control** - and `add()` means `destroy()` when you are done
- a scene's `create()` runs every time the scene starts, so "start the music in `create()` " means "start **another copy** of the music" unless it checks `this.sound.get(key)` first
- **tweens belong to a scene**; music belongs to the game. A fade made by a scene that is shutting down never finishes
- **listeners on the sound manager outlive the scene** - `TitleScene` shows the crash and the fix, exactly the Chapter 4 pattern
- `delay` **and** `seek` **are in seconds**, while nearly everything else in Phaser is milliseconds
- detune is in **cents** (100 = a semitone); rate is a multiplier; both change speed as well as pitch
- sound cannot start until the player interacts with the page: design for it with a "click to start" screen, rather than hoping

Common problems and errors

What students see	Cause	Fix
Music gets louder and louder, and sounds "phased", after going back and forth between menus	the music is added and played in <code>create()</code> every time the scene starts. (With that change to <code>Music.play</code> , <code>menu - credits - menu - credits - menu</code> leaves five <code>music_menu</code> sounds playing)	check <code>this.sound.get(key)</code> first, or keep the music in one place like <code>Music</code>
Game music still playing on the game-over screen	sounds are not stopped when their scene shuts down	stop or fade the music on purpose
Error: Audio key "music_menu" not found in cache (in the browser console), and the game stops	the sound was not loaded under that key: a misspelled key, a wrong path, or a file not copied into <code>public/assets/audio/</code> . (The loader's 404 appears as a red network error just above)	check the key, the path, and that the file is in <code>public/</code>
Property 'setVolume' does not exist on type 'BaseSound'.	the result of <code>this.sound.get(key)</code> is typed as <code>BaseSound</code> , which leaves out volume and the <code>set...</code> methods	keep the object from <code>add()</code> in a field, or tween <code>volume</code> (tweens take any object), or cast with a comment
Expected 1 arguments, but got 0. on <code>this.sound.getAll()</code>	the type definitions make the key compulsory on one of the three sound managers	pass a key, or use <code>this.sound.getAllPlaying()</code>
<code>TypeError: Cannot read properties of null (reading 'drawImage')</code> a frame after changing scene	a listener on the sound manager (e.g. "unlocked") changed a Text belonging to a scene that has shut down	remove the listener in a <code>SHUTDOWN</code> handler
A sound with <code>delay: 500</code> never plays	<code>delay</code> is in seconds - that is eight minutes	<code>delay: 0.5</code>
A fade that stops half-way; music stuck quiet	the tween was made in a scene that then shut down	make fades in the scene that is starting (as <code>Music.play</code> does)

What students see	Cause	Fix
No sound at all in the Celbridge webview or the browser, but no error	the sound is locked until the page is clicked or a key is pressed	click the game first; add a "click to start" screen
Every effect plays at once on the first click	effects were "played" while the sound was locked; Phaser holds them until it unlocks (tested: three effects played a second apart while locked all started on the click)	do not play sound before the first interaction
<code>pan</code> does nothing	the student is not wearing headphones - or the browser (Safari on iOS) has no stereo panner	headphones; never depend on <code>pan</code>
<code>this.sound.volume === 0.6</code> is false straight after setting it to 0.6	Web Audio stores volumes as 32-bit floats: it reads back as <code>0.6000000238418579</code>	never compare volumes with <code>===</code> ; keep your own setting (challenge 2)

Suggested demos

- **The duplicate music bug.** In `ch05_music_across_scenes/src/Music.ts`, change `let music: Phaser.Sound.BaseSound | null = scene.sound.get(key);` to `... = null;`. Build, and go menu - credits - menu - credits - menu. Ask the class what they hear (louder, slightly smeared music). The monitor line still shows one track, because `get` returns the *first* sound with that key - a nice lesson in itself. To count the copies, add `this.scene.sound.getAll(key).length` to the monitor's text: five. Put the line back
- **The listener leak.** In `TitleScene.ts`, remove the `this.events.once(Phaser.Scenes.Events.SHUTDOWN, ...)` block, build, open the page in a **fresh** browser window (so the sound is locked), open the console, and click. The error appears a frame after the menu starts
- **Seconds, not milliseconds.** In the sound board, change `ECHO_DELAY` to `250`. The echo "disappears". Ask why before revealing it
- **Rate vs detune.** On the sound board, press RIGHT ten times (rate 2), play a pad; press Z, then W twelve times (detune 1200), play the same pad. They sound the same: an octave

is a doubling

Discussion questions

- Why does Phaser make the sound manager game-wide, rather than giving each scene its own? (Music across scenes; one mute; one unlock.) What would be harder if it were per-scene?
- `Music` is a class with only static methods. In Java terms, what is it? Is that good OO? What would a `MusicPlayer` object stored in the registry look like instead?
- The mute setting lives in the registry *and* in `this.sound.mute`. Is that duplication? When could the two disagree? (Challenge 3 is exactly that question.)
- Why is it wrong to keep the rhythm of challenge 5 with `this.time.addEvent`?
- What makes a sound effect "feel" right? Try the random pitch with 50 cents and with 600 cents

Extension ideas

- **Audio sprites:** one file holding several sounds, with markers - `sound.addMarker({ name, start, duration })` and `sound.play(name)`. Make a marker for each half of `music_game.wav`
- **Spatial sound:** `this.sound.add(key, { source: { x, y, follow: sprite } })` and `this.sound.setListenerPosition(x, y)` - a sound that gets quieter as the player walks away
- a **settings scene** with sliders (drag a knob with the pointer) for music and effects volume, kept separate: two master volumes, one for each kind of sound
- an **analyser** that draws the music's waveform
(`this.sound.context.createAnalyser()`) - see the Phaser "audio-and-sound" notes; Web Audio only
- add sound to the Chapter 2 game: a click on start, a pop on each hit, music during play, a fanfare on the win screen, with the music stopped on the win screen

Assessment ideas

- Practical: "add a pause scene (Chapter 4) that pauses the game music while it is shown and resumes it when it closes - and plays its own quiet music meanwhile"
- Code reading: show a `create()` that does `this.sound.add("music", { loop: true }).play();` and ask what happens on the third visit to the scene, and how to fix it
- Short answer: the difference between `this.sound.play(key)` and `this.sound.add(key)`, and when to use each
- Short answer: why does `TitleScene` remove its `"unlocked"` listener at `SHUTDOWN`, when it does not remove its `"pointerdown"` listener?

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. More pads

Project: `solutions/ch05_challenge_1_more_pads` (from `ch05_sound_board`)

Goal: see that a list of sounds kept as data means adding a sound is adding a line.

`src/assets.ts`

```
{ key: "win", file: "assets/audio/win.wav" },  
  // CHALLENGE 1: four more pads - a third row. Nothing else needs to know  
  // how many there are:  
  // preload() and makePads() loop over this array  
  { key: "door", file: "assets/audio/door.wav" },  
  { key: "whoosh", file: "assets/audio/whoosh.wav" },  
  { key: "correct", file: "assets/audio/correct.wav" },  
  { key: "wrong", file: "assets/audio/wrong.wav" },  
];
```

`src/scenes/SoundBoardScene.ts`

```
this.pads.forEach((pad, index) => {  
  // CHALLENGE 1: there are only nine number keys - pads 10, 11 and 12 are  
  // click-only  
  if (index < NUMBER_KEYS.length) {  
    keyboard.on(`keydown-${NUMBER_KEYS[index]}`, () => {  
      this.playPad(pad);  
    });  
  }  
});
```

plus the four WAV files copied into `public/assets/audio/`, and the settings text moved down to make room for the third row. Tested with the harness: key 9 plays `door`; clicking pad 12 plays `wrong`.

Look for: the files really copied into `public/` (a missing file shows no error until the pad is pressed: `Audio key "door" not found in cache`); `makePads()` left alone - the layout maths already wraps into a third row. Without the `if`, pads 10-12 would listen for `"keydown-undefined"`, which never fires - harmless, but worth asking students what `NUMBER_KEYS[9]` is.

2. Turn it down

Project: `solutions/ch05_challenge_2_turn_it_down` (from `ch05_music_across_scenes`)

Goal: use the sound manager's master volume, and make something work in every scene - the same pattern as the mute button.

A new game object, made in each scene next to the mute button:

`src/objects/VolumeKey.ts`

```
const LEVELS = [1, 0.6, 0.3];
...
// nothing in the registry yet (undefined) means the first level, 100%
private level(): number {
    return this.scene.registry.get(VOLUME_LEVEL) ?? 0;
}

private nextLevel(): void {
    const level = (this.level() + 1) % LEVELS.length;
    this.scene.registry.set(VOLUME_LEVEL, level);
    this.scene.sound.volume = LEVELS[level];
    this.showLevel();
}
```

`src/scenes/MenuScene.ts` (and `CreditsScene.ts`, `GameScene.ts`)

```
new MuteButton(this, 760, 40);
// CHALLENGE 2: the master volume, next to the mute button (V changes it)
new VolumeKey(this, 725, 40);
```

Why the registry, when `this.sound.volume` is game-wide? Because reading it back does not give exactly what was set: Web Audio stores volumes as 32-bit floats, so after `this.sound.volume = 0.6`, `this.sound.volume` is `0.6000000238418579` (checked with the harness), and `LEVELS.indexOf(...)` returns -1. Keeping the *level number* in the registry avoids comparing floats - a good discussion point.

Look for: `this.sound.volume` (the master volume), not each sound's volume; the level shown in every scene; `% LEVELS.length` to go round. Accept a solution that keeps the level in a `static` field, or that reads `this.sound.volume` and picks the nearest level. A solution that loops over `this.sound.getAllPlaying()` setting each volume is wrong: sounds started later are loud again.

3. Remember the mute

Project: `solutions/ch05_challenge_3_remember_the_mute` (from `ch05_music_across_scenes`)

Goal: save a setting in the browser, and restore it when the game starts.

`src/objects/MuteButton.ts`

```
this.scene.sound.mute = muted;
// CHALLENGE 3: save it in the browser too. localStorage only holds strings:
// "true" or "false"
localStorage.setItem(MUTED_STORAGE, String(muted));
```

`src/scenes/TitleScene.ts`

```
create(): void {
    // CHALLENGE 3: the game starts here, so this is where the saved setting
    // is read back.
    // getItem gives the string that was saved - or null if nothing was, which
    // counts as "not muted"
    const muted = localStorage.getItem(MUTED_STORAGE) === "true";
    this.registry.set(MUTED, muted);
    this.sound.mute = muted;
```

Tested with the harness: mute in the menu, `location.reload()` , and the game starts with `game.sound.mute` true and the menu's button showing `sound_off` .

Look for:

- the setting restored in **both** places - the registry (for the button's picture) and `this.sound.mute` (for the sound). Restoring only the registry gives a button that says "muted" over music that plays
 - comparing with the **string** "true" . `Boolean(localStorage.getItem(...))` is a classic bug: the string "false" is truthy, so the game is always muted once anything is saved
 - a specific storage name: every page from `http://127.0.0.1:8000/` shares one `localStorage`, so "muted" would be shared by every project students serve on that port
-
-

4. Duck the music

Project: `solutions/ch05_challenge_4_duck_the_music` (from `ch05_music_across_scenes`)

Goal: `add()` plus "complete" to know when an effect ends; tweening a sound's volume from outside the `Music` class without fighting its fades.

A new method in `Music` , so the class that owns the music's volume stays the only one that tweens it:

```
src/Music.ts
```

```

public static fadeTo(scene: Phaser.Scene, key: string, fraction: number,
duration: number): void {
    const music: Phaser.Sound.BaseSound | null = scene.sound.get(key);
    if (music === null || !music.isPlaying) {
        return;
    }

    // stop the fade-in (or an earlier duck) first, so only this tween moves
the volume
    scene tweens.killTweensOf(music);
    scene tweens.add({
        targets: music,
        volume: MUSIC_VOLUME * fraction,
        duration: duration,
    });
}

```

src/scenes/GameScene.ts

```

private playPowerup(): void {
    const powerup = this.sound.add(POWERUP_KEY);
    powerup.once(Phaser.Sound.Events.COMPLETE, () => {
        powerup.destroy();
        // The sound belongs to the game, so it can finish after the player has
pressed ESC and
        // this scene has shut down. Only bring the music back up if the game is
still running -
        // otherwise the menu is already fading this music out, with its own
tweens.
        if (this.scene.isActive()) {
            Music.fadeTo(this, MUSIC_GAME_KEY, 1, DUCK_TIME);
        }
    });

    Music.fadeTo(this, MUSIC_GAME_KEY, DUCK_LEVEL, DUCK_TIME);
    powerup.play();
}

```

Tested with the harness: after ten stars the music's volume is 0.12 (a fifth of 0.6) while `powerup` plays, and 0.6 again after it ends; pressing ESC during the power-up leaves the menu music at 0.6 and the game music paused.

Look for:

- `add()` and a "complete" listener - the hint rules out `this.sound.play()`
 - `destroy()` on the power-up sound
 - `killTweensOf` before the duck: ten quick stars in the first second collide with `Music.play`'s fade-in otherwise, and the fade-in wins
 - the "has the scene gone?" question. Many students will not think of it; ask what happens if ESC is pressed during the power-up. (Without the check, nothing visibly goes wrong in this project - tested with the harness - but the listener adds a tween to a scene that has shut down, which is working by luck, not by design.) Removing the listener in a `SHUTDOWN` handler, as `TitleScene` does, is an equally good answer
-
-

5. On the beat

Project: `solutions/ch05_challenge_5_on_the_beat` (from `ch05_music_across_scenes`)

Goal: drive the game from the music's own clock (`seek`), not from a separate timer.

`src/scenes/GameScene.ts`

```

const BEATS_PER_MINUTE = 128;
const BEAT_LENGTH = 60000 / BEATS_PER_MINUTE;
...
override update(): void {
    const music: Phaser.Sound.BaseSound | null =
this.sound.get(MUSIC_GAME_KEY);
    if (music === null || !music.isPlaying) {
        return;
    }

    // BaseSound's type has no seek, although every sound has one - as in
    MusicMonitor
    const seek = (music as Phaser.Sound.WebAudioSound).seek; // seconds
    const beat = Math.floor(seek * 1000 / BEAT_LENGTH);

    // a new beat (including going from the last beat back to beat 0 when the
    music loops)
    if (beat !== this.lastBeat) {
        this.lastBeat = beat;
        this.pulse();
    }
}

```

`pulse()` is a short `yoyo` tween on the circle's scale. `lastBeat` is reset to -1 in `init()` (the scene is reused - Chapter 2's trap again).

Tested with the harness: after 5.04 seconds of music the circle had pulsed 11 times (beats 0 to 10: $5.04 \text{ s} / 0.46875 \text{ s} = 10.75$). After leaving the game for two seconds and coming back, the beat number still matched `Math.floor(seek * 1000 / 468.75)` - because it is worked out from `seek`, it cannot drift.

Look for:

- the beat worked out from `seek`, not counted. A `this.time.addEvent({ delay: 468.75, loop: true })` pulses happily - until the music pauses, the tab loses focus, or the scene restarts, when it falls out of step. Ask students who did that to test with ESC and back
- seconds versus milliseconds: `seek` is in seconds, `BEAT_LENGTH` in milliseconds
- nothing happens while the music is not playing (for example during the first frame, before `Music.play` has made it)

- `music_game.wav` is exactly 15 seconds, which is exactly 32 beats, so the loop does not break the rhythm. A sharp student will ask what would happen if it were not

6. Jukebox

Project: `solutions/ch05_challenge_6_jukebox` (from `ch05_sound_board`)

Goal: several sound objects, `"complete"` to chain them, and knowing that `stop()` does not fire `"complete"`.

`src/scenes/SoundBoardScene.ts`

```
// CHALLENGE 6: one sound object per track, NOT looping - so each one fires
// "complete" when
// it reaches its end, and that starts the next. ("complete" does not fire
// when a sound is
// stopped with stop(), so skipping with K cannot set off a second skip.)
this.tracks = MUSIC_TRACKS.map((track) => {
  const sound = this.sound.add(track.key, { loop: false, volume:
MUSIC_VOLUME });
  sound.on(Phaser.Sound.Events.COMPLETE, () => {
    this.nextTrack();
  });
  return sound;
});
```

```
private get music(): GameSound {
  return this.tracks[this.trackIndex];
}
...
private nextTrack(): void {
  this.music.stop();
  this.trackIndex = (this.trackIndex + 1) % this.tracks.length;
  this.music.setVolume(this.volume *
MUSIC_VOLUME).setRate(this.rate).setDetune(this.detune);
  this.music.play();
}
```

The **getter** (`get music()`) is the neat trick: every existing line that says `this.music` - play, pause, stop, loop, skip, the progress bar - now means "the current track", with no other change. It is TypeScript's version of a Java `getMusic()` method, used without brackets.

Tested with the harness: K moves `music_menu` to `music_game` ; seeking to 0.3 seconds before the end moves on to `music_action` , then round to `music_menu` ; only one track is ever playing.

Look for:

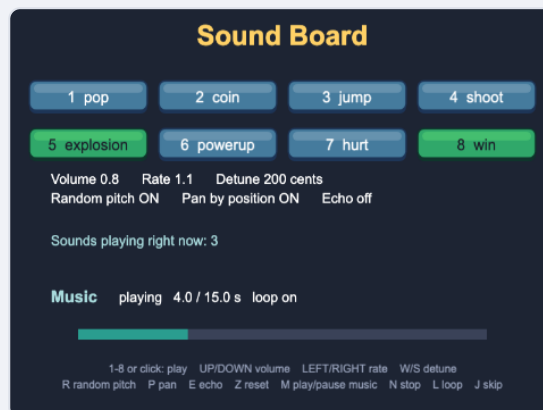
- `on` (not `once`) for `"complete"` - each track completes every time round the playlist
- the settings copied onto the next track (each track is a separate sound object that has not seen the volume, rate or detune keys)
- one track playing at a time: a common bug is starting the next track without stopping the current one on K, so two play at once
- a solution that uses a single sound object and `this.sound.add` again for each track (destroying the old one) is also fine - check it destroys

Slides

Chapter 5

Audio

Sound effects, music, and the rules of the browser



1 / 20

Today

- loading sounds; playing them with a **config**
- `play()` - fire and forget - or `add()` - keep hold
- a sound's states, and its "complete" event
- the sound manager belongs to the **game**
- cross-fading music with tweens; mute everywhere
- the browser's **autoplay** rule

2 / 20

Loading sounds

```
preload(): void {  
  for (const pad of PAD_SOUNDS) {  
    this.load.audio(pad.key, pad.file);  
  }  
  this.load.audio(MUSIC_KEY, MUSIC_FILE);  
}
```

- like pictures: a key and a file; loaded once, for every scene
- an **array** of files lets the browser pick a format it can play: `["theme.ogg", "theme.mp3"]`

3 / 20

Fire and forget

```
this.sound.play(COIN_KEY, {  
  detune: Phaser.Math.Between(-RANDOM_DETUNE, RANDOM_DETUNE),  
  pan: (star.x - centreX) / centreX * MAX_PAN,  
});
```

- makes a sound, plays it, **destroys** it when it ends
- config: `volume` `rate` `detune` `pan` `loop` `delay` `seek` `mute`
- leave a setting out and it keeps its default

4 / 20

Rate, detune - and seconds

- `rate: 2` - twice as fast, an octave higher
- `detune: 1200` - also an octave higher (cents: 100 = a semitone)
- both change speed **and** pitch; Phaser multiplies them

```
const ECHO_DELAY = 0.25; // seconds (SoundConfig delays are in seconds, not milliseconds)
```

`delay` and `seek` are in **seconds**. `delay: 250` waits four minutes.

5 / 20

Less mechanical

The same sound, again and again, sounds like a machine.

- a small **random detune** each time
- **pan** by position: -1 left, 0 middle, 1 right (headphones!)

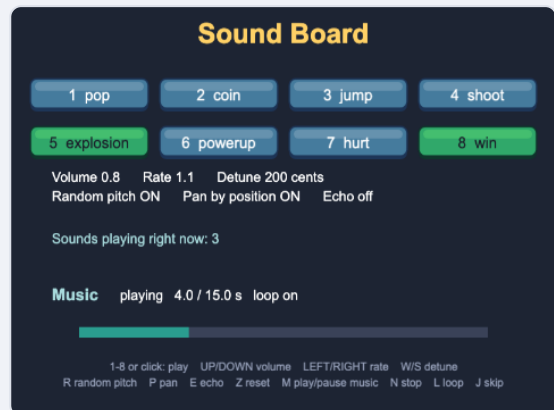
```
pan: this.panByPosition ? this.panFor(pad.x) : 0,
```

Pan is ignored where there is no stereo panner (iPhones, iPads): a finishing touch only.

6 / 20

Try it: Sound Board

- 1-8 play; arrows change volume and rate; W/S detune
- R random pitch, P pan, E echo
- M / N / L / J: the music - play and pause, stop, loop, skip
- find where each setting goes into a `SoundConfig`



7 / 20

Keep hold: `add`

```
const sound = this.scene.sound.add(this.soundKey);

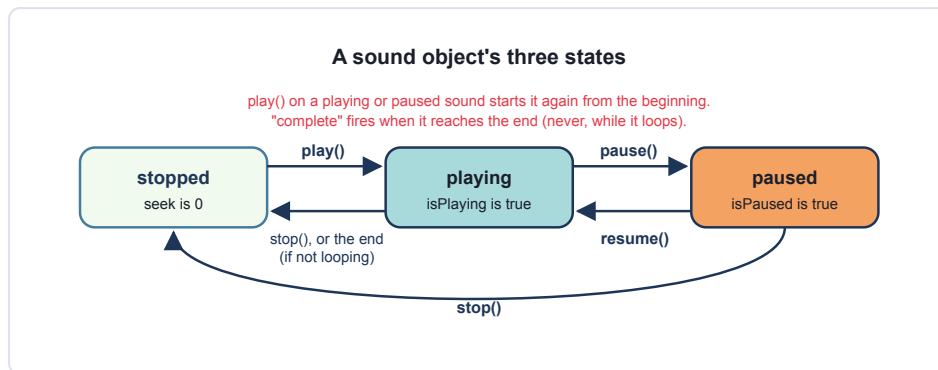
sound.once(Phaser.Sound.Events.COMPLETE, () => {
  this.playing = this.playing - 1;
  this.showState();
  sound.destroy();
});

sound.play(config);
```

- `add()` gives you the sound object - and does **not** start it
- it stays in the sound manager until **you** `destroy()` it

8 / 20

Three states



`play()` on a playing or paused sound starts it **again from the beginning**

9 / 20

Changing a playing sound

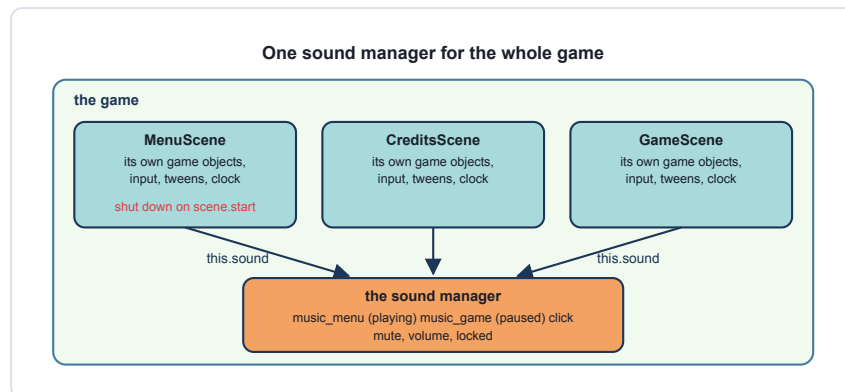
```
if (this.music.isPlaying) {
  this.music.pause();
} else if (this.music.isPaused) {
  this.music.resume();
} else {
  this.music.play();
}
```

```
this.music.setVolume(MUSIC_VOLUME).setRate(1).setDetune(0);
```

`setLoop` `setPan` `setSeek` - and `seek` , `duration` to read back

10 / 20

One sound manager



Sounds **keep playing** when their scene shuts down.

11 / 20

The duplicate music bug

```
// in create() - runs EVERY time the scene starts
this.sound.add(MUSIC_MENU_KEY, { loop: true }).play();
```

menu - credits - menu - credits - menu: **five** copies playing

Ask first:

```
let music: Phaser.Sound.BaseSound | null = scene.sound.get(key);
```

`null` if there is no sound with that key

12 / 20

Music.play(this, key)

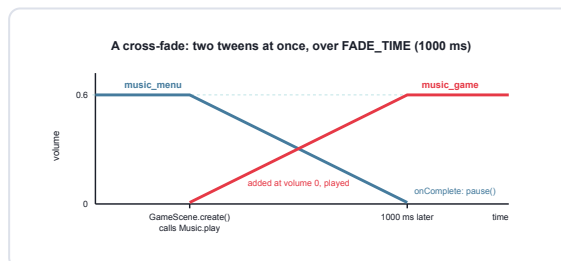
```
if (music === null) {  
  music = scene.sound.add(key, { loop: true, volume: 0 });  
  music.play();  
} else if (music.isPaused) {  
  music.resume();  
} else if (!music.isPlaying) {  
  music.play();  
}
```

- every scene names the music it wants, in `create()`
- already playing? leave it alone
- every **other** track fades out, and is paused

13 / 20

Cross-fading - whose tweens?

```
scene.tweens.add({  
  targets: music,  
  volume: MUSIC_VOLUME,  
  duration: FADE_TIME,  
});
```



Tweens belong to a **scene**, and die with it: make fades in the scene that is **starting**

14 / 20

Mute, everywhere

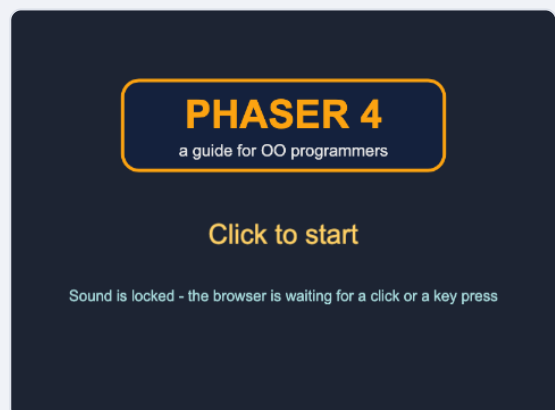
```
private toggle(): void {  
    const muted = !this.isMuted();  
    this.scene.registry.set(MUTED, muted);  
    this.scene.sound.mute = muted;  
    this.showState();  
}
```

- `this.sound.mute` , `this.sound.volume` - the whole game
- a `MuteButton` in every scene; one setting in the registry

15 / 20

The autoplay rule

- no sound until the player clicks, taps or presses a key
- `this.sound.locked` ; the `"unlocked"` event
- sounds played while locked wait - then all start at once
- so: a **"click to start"** screen



16 / 20

A listener that outlived its scene

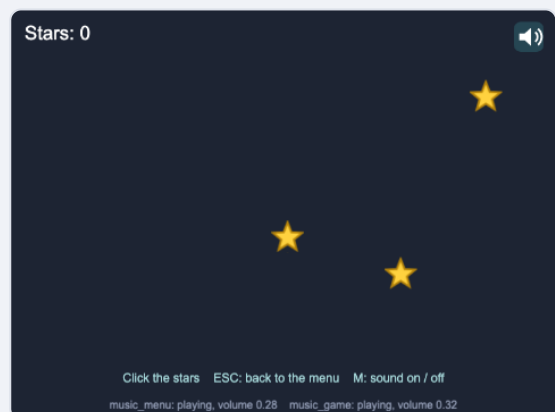
```
const onUnlocked = (): void => {
  status.setText("Sound is unlocked");
};
this.sound.once(Phaser.Sound.Events.UNLOCKED, onUnlocked);
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.sound.off(Phaser.Sound.Events.UNLOCKED, onUnlocked);
});
```

Without the `SHUTDOWN` part: `TypeError: Cannot read properties of null (reading 'drawImage')`

17 / 20

Try it: Music Across Scenes

- menu - credits - menu: the music carries on
- menu - game: watch the cross-fade on the bottom line
- M in any scene; then change scene
- leave the game quickly: does anything get stuck?



18 / 20

Summary

- `this.load.audio(key, file) ; this.sound.play(key, config)` - fire and forget
- `this.sound.add(key, config)` - keep it; `play` `pause` `resume` `stop` `set...` ; `destroy()`
- `"complete"` when a sound ends; `delay` and `seek` in seconds
- one sound manager for the game: check `this.sound.get(key)` before starting music
- `tween volume` to fade - in the scene that is starting
- `this.sound.mute` , `this.sound.volume` ; `locked` until the first click

19 / 20

Challenges

1. **More pads** - a third row of four
2. **Turn it down** - V: 100%, 60%, 30%, in every scene
3. **Remember the mute** - survives a refresh (`localStorage`)
4. **Duck the music** - quieter under the power-up
5. **On the beat** - a circle that pulses at 128 bpm, in time after pauses
6. **Jukebox** - three tracks in turn; K for the next

Next: **Chapter 6 - Scoring**

20 / 20